

DBMS Lab — Step-by-Step Walkthrough (All 3 Labs)

LAB 1: TRANSACTIONS (COMMIT, ROLLBACK, SAVEPOINT)

Step 1: Environment Setup

1. Open **XAMPP Control Panel**.
2. Click **Start** next to **Apache**.
3. Click **Start** next to **MySQL**.
4. Click **Admin** next to MySQL to launch **phpMyAdmin** (or open the MySQL CLI instead).
5. Go to the **SQL** tab.
6. Run:

```
sql

SET autocommit = 0;
```

This disables MySQL's default auto-save behavior so you can control commits manually.

Step 2: Create the Database and Table

1. In the SQL tab, run:

```
sql

CREATE DATABASE LabTransactions;
USE LabTransactions;
```

2. Create the table — note `ENGINE=InnoDB` is required (transactions don't work on MyISAM):

```
sql

CREATE TABLE Accounts (
  account_id INT PRIMARY KEY,
  name VARCHAR(50),
  balance DECIMAL(10,2)
) ENGINE=InnoDB;
```

3. Insert starting data:

sql

```
INSERT INTO Accounts VALUES (1, 'Alice', 1000.00);  
INSERT INTO Accounts VALUES (2, 'Bob', 500.00);
```

4. Verify:

sql

```
SELECT * FROM Accounts;
```

You should see Alice = 1000.00, Bob = 500.00.

Step 3: Exercise 1 — Successful Transaction (COMMIT)

Goal: transfer \$200 from Alice to Bob and save it permanently.

1. Start the transaction:

sql

```
START TRANSACTION;
```

2. Deduct from Alice:

sql

```
UPDATE Accounts SET balance = balance - 200.00 WHERE account_id = 1;
```

3. Add to Bob:

sql

```
UPDATE Accounts SET balance = balance + 200.00 WHERE account_id = 2;
```

4. Save permanently:

sql

```
COMMIT;
```

5. Verify:

sql

```
SELECT * FROM Accounts;
```

✓ Expected: Alice = 800.00, Bob = 700.00

Step 4: Exercise 2 — Failed Transaction (ROLLBACK)

Goal: simulate Alice sending \$300 to Bob, but the system fails before Bob receives it — undo everything.

1. Start a new transaction:

sql

```
START TRANSACTION;
```

2. Deduct from Alice:

sql

```
UPDATE Accounts SET balance = balance - 300.00 WHERE account_id = 1;
```

3. Simulate the failure — do **not** run the "add to Bob" step. Instead cancel the transaction:

sql

```
ROLLBACK;
```

4. Verify:

sql

```
SELECT * FROM Accounts;
```

✓ Expected: Alice stays 800.00, Bob stays 700.00 — the deduction was undone.

Step 5: Exercise 3 — Partial Rollback (SAVEPOINT)

Goal: give Alice a \$100 bonus (keep it), then accidentally give Bob a \$500 bonus (undo just that part).

1. Start the transaction:

sql

```
START TRANSACTION;
```

2. Give Alice a bonus:

sql

```
UPDATE Accounts SET balance = balance + 100.00 WHERE account_id = 1;
```

3. Create a checkpoint right after that update:

sql

```
SAVEPOINT BonusAdded;
```

4. Give Bob a bonus by mistake:

sql

```
UPDATE Accounts SET balance = balance + 500.00 WHERE account_id = 2;
```

5. Undo only the mistake (rolls back to the checkpoint, discarding Bob's bonus):

sql

```
ROLLBACK TO SAVEPOINT BonusAdded;
```

6. Save everything that remains (Alice's bonus):

sql

```
COMMIT;
```

7. Verify:

sql

```
SELECT * FROM Accounts;
```

✓ Expected: **Alice = 900.00** (bonus kept), **Bob = 700.00** (mistake undone).

LAB 2: NORMALIZATION (1NF → 2NF → 3NF)

Step 1: Environment Setup

1. Download XAMPP from Apache Friends and run the installer with default components (if not already installed).
2. Launch XAMPP Control Panel → start **Apache** and **MySQL**.
3. Go to `localhost` in your browser → click **phpMyAdmin**.
4. Log in with your MySQL credentials if prompted.
5. Create a new database for this lab (e.g. `LabNormalization`) and select it.

Step 2: Start From the Unnormalized Table

Picture (or actually create, to see the problem firsthand) a single flat table:

StudentID	StudentName	Courses	Instructors	Departments
1	Alice Johnson	Math, Physics, Chemistry	Dr. Smith, Dr. Brown, Dr. Green	Science, Science, Science
2	Bob Lee	English, History	Prof. White, Prof. Black	Humanities, Humanities
3	Carol Davis	Math, Biology	Dr. Smith, Dr. Yellow	Science, Science

Identify the problems out loud before moving on (you'll likely be asked to list these):

- Repeating groups (multiple courses in one cell) → violates 1NF
- Update anomaly (changing "Dr. Smith" requires editing multiple rows)
- Insertion anomaly (can't add a course without a student)
- Deletion anomaly (deleting a student erases course info)
- Redundancy (Department repeats for every student-course pair)

Step 3: Apply First Normal Form (1NF)

Rule: every cell = one atomic value; no repeating groups.

1. Break the flat table into one row per student-course pair.

2. Build the Students table:

StudentID	StudentName
1	Alice Johnson
2	Bob Lee
3	Carol Davis

3. Build the StudentCourses table (1NF), primary key = (StudentID, CourseName):

StudentID	CourseName	Instructor	Department
1	Math	Dr. Smith	Science
1	Physics	Dr. Brown	Science
1	Chemistry	Dr. Green	Science
2	English	Prof. White	Humanities
2	History	Prof. Black	Humanities
3	Math	Dr. Smith	Science
3	Biology	Dr. Yellow	Science

4. Confirm: ✓ atomic cells, ✓ no repeating groups, ✓ composite PK assigned.
Remaining issue to flag: partial and transitive dependencies still exist.

Step 4: Apply Second Normal Form (2NF)

Rule: every non-key column must depend on the *entire* composite key, not just part of it.

- Test each column in StudentCourses against the key (StudentID, CourseName):
 - Instructor → depends only on CourseName → **partial dependency, violates 2NF**
 - Department → depends only on CourseName → **partial dependency, violates 2NF**
- Decompose: move Instructor and Department into their own Courses table.
Courses table:

CourseName	Instructor	Department
Math	Dr. Smith	Science
Physics	Dr. Brown	Science
Chemistry	Dr. Green	Science
English	Prof. White	Humanities
History	Prof. Black	Humanities
Biology	Dr. Yellow	Science

Enrollments table (2NF), PK = (StudentID, CourseName):

StudentID	CourseName
1	Math
1	Physics
1	Chemistry
2	English
2	History
3	Math
3	Biology

- Confirm: ✓ every non-key column now depends on the full key. **Remaining issue to flag:** Department depends on Instructor (a non-key column) inside Courses, not directly on CourseName → transitive dependency.

Step 5: Apply Third Normal Form (3NF)

Rule: a non-key column must depend only on the primary key — never on another non-key column.

- In Courses, test: does Department depend on CourseName directly, or through Instructor? → Through Instructor → **transitive dependency, violates 3NF**.
- Decompose further: split Instructors out of Courses. Instructors table:

Instructor	Department
Dr. Smith	Science
Dr. Brown	Science
Dr. Green	Science
Prof. White	Humanities
Prof. Black	Humanities
Dr. Yellow	Science

Courses table (3NF):

CourseName	Instructor (FK)
Math	Dr. Smith
Physics	Dr. Brown
Chemistry	Dr. Green
English	Prof. White
History	Prof. Black
Biology	Dr. Yellow

Enrollments table (3NF), PK = (StudentID, CourseName):

StudentID (FK)	CourseName (FK)
1	Math
1	Physics
...	...

3. Confirm: ✓ no transitive dependencies, ✓ every non-key column depends only on its table's primary key, ✓ zero redundancy, ✓ full referential integrity via FKs.

Step 6: Implement the Final Schema in MySQL

1. Run each **CREATE TABLE** in order — parents before children (Students/Instructors

before Courses, Courses before Enrollments):

sql

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    StudentName VARCHAR(100)  
);  
  
CREATE TABLE Instructors (  
    Instructor VARCHAR(100) PRIMARY KEY,  
    Department VARCHAR(100)  
);  
  
CREATE TABLE Courses (  
    CourseName VARCHAR(100) PRIMARY KEY,  
    Instructor VARCHAR(100),  
    FOREIGN KEY (Instructor) REFERENCES Instructors(Instructor)  
);  
  
CREATE TABLE Enrollments (  
    StudentID INT,  
    CourseName VARCHAR(100),  
    PRIMARY KEY (StudentID, CourseName),  
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),  
    FOREIGN KEY (CourseName) REFERENCES Courses(CourseName)  
);
```

2. Populate each table with the data shown in Step 5 using `INSERT INTO`.

Step 7: Test and Verify

1. Reconstruct the original flat report using a JOIN:

sql

```
SELECT S.StudentName, C.CourseName, C.Instructor, I.Department  
FROM Students S  
INNER JOIN Enrollments E ON S.StudentID = E.StudentID  
INNER JOIN Courses C ON E.CourseName = C.CourseName  
INNER JOIN Instructors I ON C.Instructor = I.Instructor;
```

2. Audit: check for orphaned rows, confirm foreign keys are enforced, confirm no column repeats data unnecessarily.
3. Optimize (conceptually): note that foreign key columns are good candidates for indexing (ties into Lab 3).

LAB 3: CASE STUDY — SCHEMA DESIGN, FK CONSTRAINTS, INDEXING

Part A: Build the Users / Products / Orders Schema

Step 1: Create the Tables

sql

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(150) NOT NULL UNIQUE  
);  
  
CREATE TABLE products (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  product_name VARCHAR(150) NOT NULL,  
  price DECIMAL(10, 2) NOT NULL  
);  
  
CREATE TABLE orders (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  user_id INT,  
  product_id INT,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
  FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE RESTRICT  
);
```

Note *why* each FK rule was chosen as you type it: `user_id` uses `CASCADE` so a deleted user's orders don't become orphaned; `product_id` uses `RESTRICT` so historical sales records can't be broken by deleting a product.

Step 2: Insert Sample Data

sql

```
INSERT INTO users (name, email) VALUES
('Alice Smith', 'alice@example.com'),
('Bob Jones', 'bob@example.com');

INSERT INTO products (product_name, price) VALUES
('Wireless Mouse', 25.00),
('Mechanical Keyboard', 85.50),
('Gaming Monitor', 299.99);

INSERT INTO orders (user_id, product_id) VALUES
(1, 1), -- Alice buys Mouse
(2, 2), -- Bob buys Keyboard
(2, 3); -- Bob buys Monitor
```

Step 3: Fetch Combined Data (JOIN)

```
sql

SELECT
  orders.id AS order_number,
  users.name AS customer_name,
  products.product_name,
  products.price,
  orders.order_date
FROM orders
INNER JOIN users ON orders.user_id = users.id
INNER JOIN products ON orders.product_id = products.id;
```

Confirm the output shows 3 rows: Alice/Mouse, Bob/Keyboard, Bob/Monitor.

Step 4: Test A — FK Restriction ((ON DELETE RESTRICT))

1. Try to delete a product that has an existing order:

```
sql

DELETE FROM products WHERE id = 1;
```

2. Observe: MySQL **blocks** this with error `#1451 - Cannot delete or update a parent row: a foreign key constraint fails`.
3. Explain why: this protects sales history — you can't delete a product that's referenced by past orders.

Step 5: Test B — Cascading Delete ((ON DELETE CASCADE))

1. Delete a user entirely:

sql

```
DELETE FROM users WHERE id = 1;
```

2. Observe: MySQL **allows** this.
3. Re-run the JOIN query from Step 3 — Alice's order (order #1) is now automatically gone too.
4. Explain why: this prevents orphaned order records pointing to a user that no longer exists.

Part B: Indexing

Step 1: Design the Table (note which columns get auto-indexed)

sql

```
CREATE TABLE inventory (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  sku VARCHAR(50) NOT NULL UNIQUE,  
  product_name VARCHAR(150) NOT NULL,  
  category_unindexed VARCHAR(100) NOT NULL,  
  stock_count INT NOT NULL,  
  INDEX idx_product_name (product_name)  
);
```

- `id` → auto-indexed via PRIMARY KEY
- `sku` → auto-indexed via UNIQUE
- `product_name` → explicitly indexed
- `category_unindexed` → deliberately left unindexed, for comparison

Step 2: Populate Mock Data

sql

```
INSERT INTO inventory (sku, product_name, category_unindexed, stock_count) VALUES
('SKU-001', 'Logitech Wireless Mouse M325', 'Electronics', 150),
('SKU-002', 'Razer DeathAdder Gaming Mouse', 'Electronics', 85),
('SKU-003', 'Dell UltraSharp 27 Inch Monitor', 'Computers', 40),
('SKU-004', 'Apple MacBook Pro 16', 'Computers', 25),
('SKU-005', 'Samsung Galaxy S24 Ultra', 'Smartphones', 60),
('SKU-006', 'Anker PowerCore 20K Power Bank', 'Accessories', 200);
```

Step 3: Compare an Unindexed vs Indexed Search

Scenario A — search the unindexed column:

sql

```
EXPLAIN SELECT * FROM inventory WHERE category_unindexed = 'Computers';
```

Check the results panel:

- `type` = `ALL` → full table scan
- `possible_keys` = `NULL` → no index available
- `rows` = `6` → every row checked

Scenario B — search the indexed column:

sql

```
EXPLAIN SELECT * FROM inventory WHERE product_name = 'Apple MacBook Pro 16';
```

Check the results panel:

- `type` = `ref` → index lookup
- `key` = `idx_product_name` → the index actually used
- `rows` = `1` → went straight to the target row

Step 4: Create a Composite (Multi-Column) Index

sql

```
ALTER TABLE inventory ADD INDEX idx_name_stock (product_name, stock_count);
```

Step 5: Analyze the Optimization

sql

```
EXPLAIN SELECT * FROM inventory
WHERE product_name = 'Razer DeathAdder Gaming Mouse' AND stock_count > 0;
```

Observe that MySQL satisfies **both** conditions ((product_name) AND (stock_count)) in a single lookup pass using (idx_name_stock), instead of needing a separate scan for the second condition.

Checklist — run through this the night before

- ☐ Lab 1: recreate Accounts table, run all 3 transaction exercises, predict balances before checking
- ☐ Lab 2: redraw the unnormalized table, then manually normalize it to 1NF → 2NF → 3NF on paper
- ☐ Lab 2: type out the final 4 (CREATE TABLE) statements from memory
- ☐ Lab 3: recreate users/products/orders, run both DELETE tests, explain RESTRICT vs CASCADE out loud
- ☐ Lab 3: run both EXPLAIN queries, be able to state what (type), (key), (possible_keys), (rows) mean